LAB: GPIO Digital InOut

LAB: GPIO Digital InOut

Date: 2024-09-17

Author/Partner: HWANG MooYeol

Github: [-]

Demo Video: LAB1 Demo Video

PDF version: LAB1 PDF

Introduction

In this lab, you are required to create a simple program that toggle multiple LEDs with a push-button input. Create HAL drivers for GPIO digital in and out control and use your library.

You must submit

- LAB Report (*.pdf)
- Zip source files(main*.c, ecRCC.h, ecGPIO.h etc...).
  - Only the source files. Do not submit project files

Requirement

Hardware

- MCU
  - NUCLEO-F411RE
- Actuator/Sensor/Others:
  - LEDs x 4
  - Array resistor, breadboard

Software

- Keil uVision, CMSIS, EC_HAL library

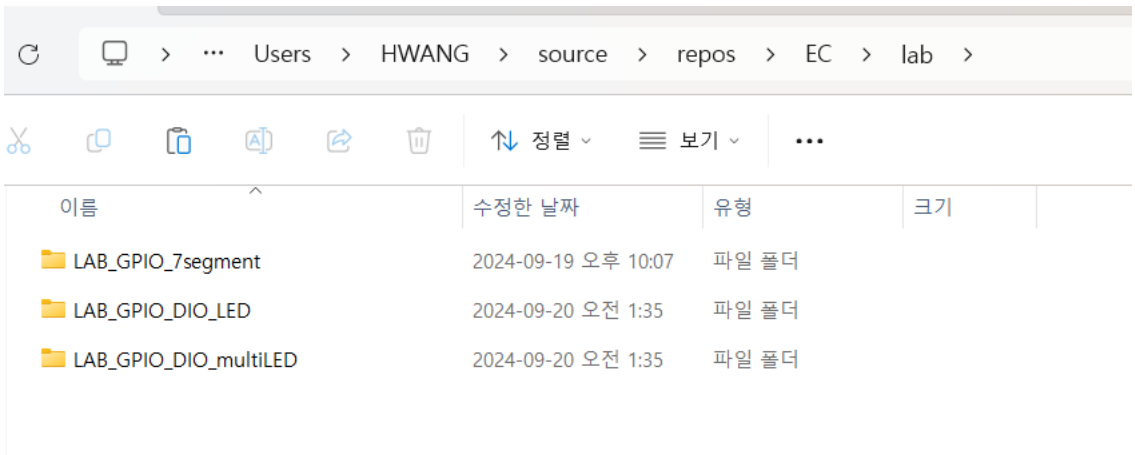
Problem 0: STM-Arduino

We are going to create a simple program that turns LED(LD2) on and off by pressing the user button(BT1), using Arduino Syntax

GPIO Digital In/Out

Create a new project under the directory `\repos\EC\LAB\`

- The project folder name is "**\LAB_GPIO_DIO_LED".**



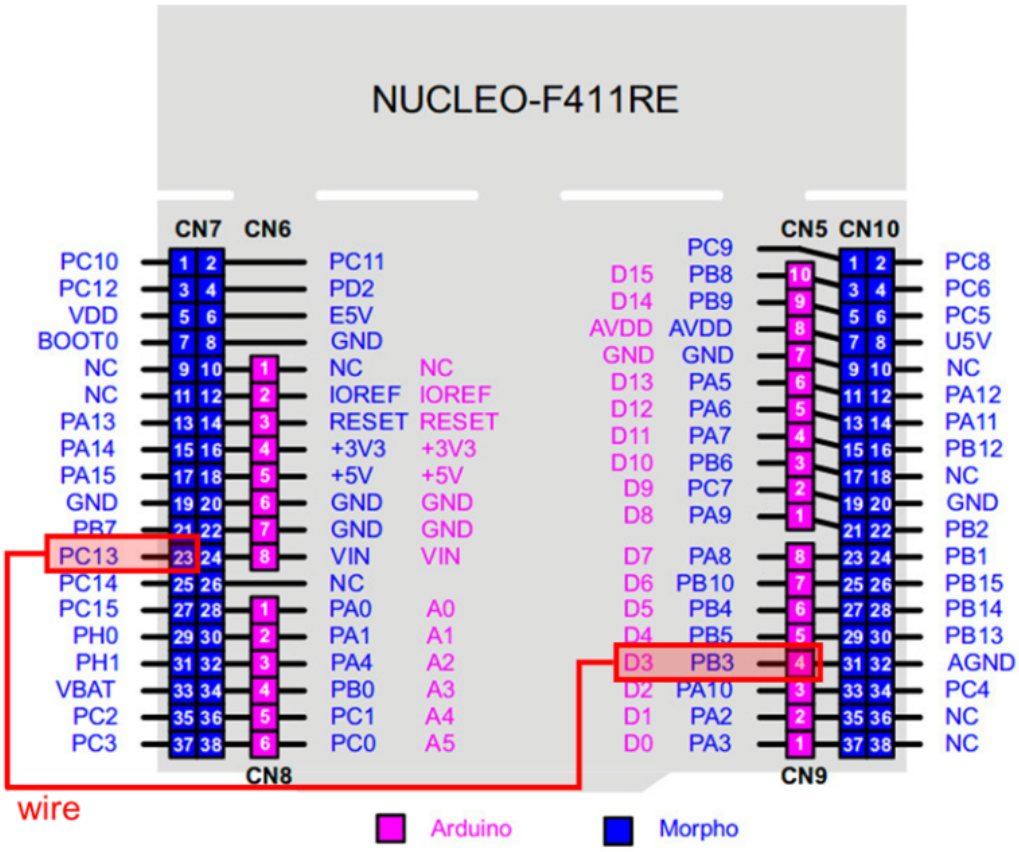
Example code

Open *Arduino IDE* and Create a new program named as '**TU_arduino_GPIO_LED_button.ino**'.

Write the following source code: [source code](#).

```
const int btnPin = 3;
const int ledPin = 13;

int btnState = HIGH;

void setup() {
  pinMode(ledPin, OUTPUT);
  pinMode(btnPin, INPUT);
}

void loop() {
  btnState = digitalRead(btnPin);

  if (btnState == HIGH)
    digitalWrite(ledPin, LOW);

  else
    digitalWrite(ledPin, HIGH);

}
```

The user button pin is `PC13`, but this pin cannot be used in arduino. So, you should connect `PC13` to `pinName` `D3` by using wire.



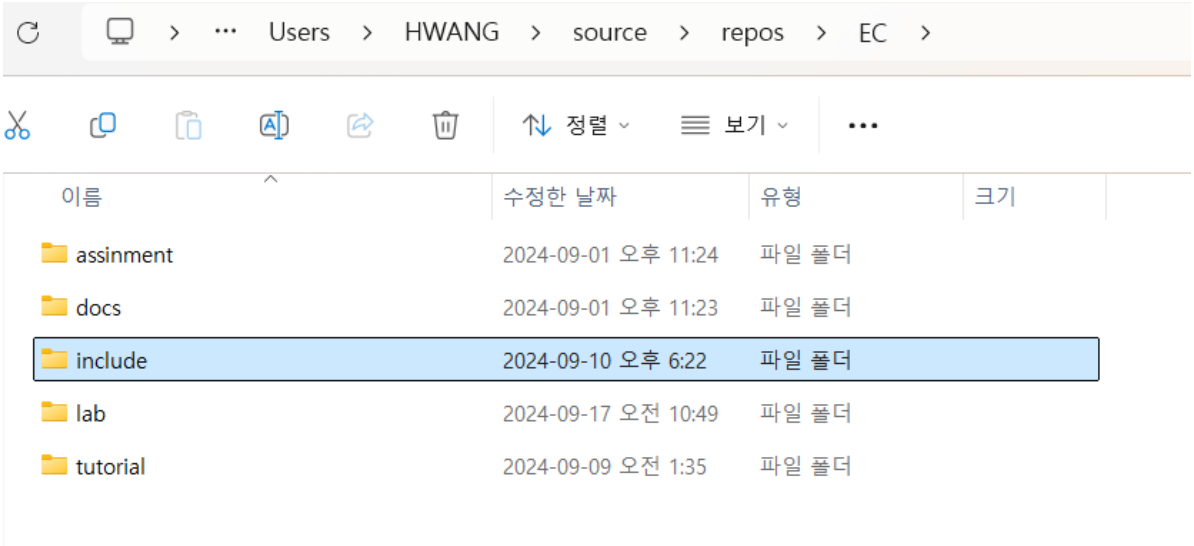
Click on **upload** button. Push the reset button(black) and check the performance.

The LED(LD2) should be turned on when the button is pressed.

Problem 1: Create EC_HAL library

Procedure

Create the library directory `\repos\EC\lib\`.



ecRCC2.h

```
#ifndef __EC_RCC2_H
#define __EC_RCC2_H

#ifdef __cplusplus
 extern "C" {
#endif /* __cplusplus */

//#include "stm32f411xe.h"

void RCC_HSI_init(void);
void RCC_PLL_init(void);
void RCC_GPIOA_enable(void);
void RCC_GPIOB_enable(void);
void RCC_GPIOC_enable(void);
//void RCC_GPIO_enable(GPIO_TypeDef * GPIOx);

extern int EC_SYSCL;

#ifdef __cplusplus
}
#endif /* __cplusplus */

#endif // __EC_RCC2_H
```

ecGPIO2_student.h

```
#ifndef __ECGPIO2_H
#define __ECGPIO2_H

#include "stm32f411xe.h"
#include "ecRCC2.h"
#include "ecPinNames.h"


#define CLEAR_2BIT(PORT,PIN)  ((PORT) &= ~(3<<2*(PIN)))

typedef enum {INPUT = 0, OUTPUT, AF, ANALOG}GPIO_MODE;

/*
#define INPUT  0x00
#define OUTPUT 0x01
#define AF     0x02
#define ANALOG 0x03
*/

typedef enum {PUSH_PULL = 0, OPEN_DRAIN = 1}GPIO_TYPE;

typedef enum { LOW_SPEED    = 0b00,
                             MEDIUM_SPEED = 0b01,
                             FAST_SPEED   = 0b10,
                             HIGH_SPEED   = 0b11
                          }GPIO_SPEED;

#define NO_PUPD 0x00
```

```
#define PULL_UP 0x01
#define PULL_DOWN 0x02

#define HIGH 1
#define LOW  0

#define LED_PIN    //Find LED Port&Pin and Fill the blank
#define BUTTON_PIN PC_13//Find BTN Port&Pin and Fill the blank

#ifdef __cplusplus
 extern "C" {
#endif /* __cplusplus */

void GPIO_init(PinName_t pinName, uint32_t mode);
void GPIO_write(PinName_t pinName, int Output);
int  GPIO_read(PinName_t pinName);
void GPIO_mode(PinName_t pinName, uint32_t mode);
void GPIO_ospeed(PinName_t pinName, int speed);
void GPIO_otype(PinName_t pinName, int type);
void GPIO_pupd(PinName_t pinName, int pupd);
void GPIO_toggle(PinName_t pinName);
void delay_ms(int num);



#ifdef __cplusplus
}
#endif /* __cplusplus */

#endif // __ECGPIO2_H
```

ecRCC2.c

```
#include "stm32f4xx.h"
#include "ecRCC2.h"

volatile int EC_SYSCLK=16000000;

void RCC_HSI_init() {
    // Enable High Speed Internal Clock (HSI = 16 MHz)
  //RCC->CR |= ((uint32_t)RCC_CR_HSION);
    RCC->CR |= 0x00000001U;

  // wait until HSI is ready
  //while ( (RCC->CR & (uint32_t) RCC_CR_HSIRDY) == 0 ) {;}
    while ( (RCC->CR & 0x00000002U) == 0 ) {;}

  // Select HSI as system clock source
  RCC->CFGR &= (uint32_t)(~RCC_CFGR_SW);                                // not
essential
  RCC->CFGR |= (uint32_t)RCC_CFGR_SW_HSI;                               //00:
HSI16 oscillator used as system clock

    // Wait till HSI is used as system clock source
  while ((RCC->CFGR & (uint32_t)RCC_CFGR_SWS) != 0 );
```

```
    //EC_SYSTEM_CLK=16000000;
        //EC_SYSCLK=16000000;
        EC_SYSCLK=16000000;
}

void RCC_PLL_init() {
    // To correctly read data from FLASH memory, the number of wait states
(LATENCY)
  // must be correctly programmed according to the frequency of the CPU clock
  // (HCLK) and the supply voltage of the device.
    FLASH->ACR &= ~FLASH_ACR_LATENCY;
    FLASH->ACR |=  FLASH_ACR_LATENCY_2WS;

    // Enable the Internal High Speed oscillator (HSI)
    RCC->CR |= RCC_CR_HSION;
    while((RCC->CR & RCC_CR_HSIRDY) == 0);

    // Disable PLL for configuration
    RCC->CR    &= ~RCC_CR_PLLON;

    // Select clock source to PLL
    RCC->PLLCFGR &= ~RCC_PLLCFGR_PLLSRC;        // Set source for PLL: clear
bits
    RCC->PLLCFGR |= RCC_PLLCFGR_PLLSRC_HSI; // Set source for PLL: 0 =HSI, 1 =
HSE

    // Make PLL as 84 MHz
    // f(VCO clock) = f(PLL clock input) * (PLLN / PLLM) = 16MHz * 84/8 = 168
MHz
    // f(PLL_R) = f(VCO clock) / PLLP = 168MHz/2 = 84MHz
    RCC->PLLCFGR = (RCC->PLLCFGR & ~RCC_PLLCFGR_PLLN) | 84U << 6;
    RCC->PLLCFGR = (RCC->PLLCFGR & ~RCC_PLLCFGR_PLLM) | 8U ;
    RCC->PLLCFGR &= ~RCC_PLLCFGR_PLLP;  // 00: PLLP = 2, 01: PLLP = 4, 10: PLLP
= 6, 11: PLLP = 8


    // Enable PLL after configuration
    RCC->CR   |= RCC_CR_PLLON;
    while((RCC->CR & RCC_CR_PLLRDY)>>25 != 0);

    // Select PLL as system clock
    RCC->CFGR &= ~RCC_CFGR_SW;
    RCC->CFGR |= RCC_CFGR_SW_PLL;

    // Wait until System Clock has been selected
    while ((RCC->CFGR & RCC_CFGR_SWS) != 8UL);

    // The maximum frequency of the AHB and APB2 is 100MHz,
    // The maximum frequency of the APB1 is 50 MHz.
    RCC->CFGR &= ~RCC_CFGR_HPRE;        // AHB prescaler = 1; SYSCLK not divided
(84MHz)
    RCC->CFGR &= ~RCC_CFGR_PPRE1;       // APB high-speed prescaler (APB1) = 2,
HCLK divided by 2 (42MHz)
    RCC->CFGR |=  RCC_CFGR_PPRE1_2;
    RCC->CFGR &= ~RCC_CFGR_PPRE2;       // APB high-speed prescaler (APB2) = 1,
HCLK not divided    (84MHz)

    EC_SYSCLK=84000000;
```

```
}


void RCC_GPIOA_enable()
{
    // RCC Peripheral Clock Enable Register
    RCC->AHB1ENR |= RCC_AHB1ENR_GPIOAEN;
}

void RCC_GPIOB_enable()
{
    // RCC Peripheral Clock Enable Register
    RCC->AHB1ENR |= RCC_AHB1ENR_GPIOBEN;
}

void RCC_GPIOC_enable()
{
    // RCC Peripheral Clock Enable Register
    RCC->AHB1ENR |= RCC_AHB1ENR_GPIOCEN;
}
```

ecGPIO2_student.c

```
#include "stm32f4xx.h"
#include "stm32f411xe.h"
#include "ecGPIO2_student.h"

void GPIO_init(PinName_t pinName, uint32_t mode){
    GPIO_TypeDef * Port;
    unsigned int pin;
    ecPinmap(pinName, &Port, &pin);

    // mode  : Input(0), Output(1), AlterFunc(2), Analog(3)
    if (Port == GPIOA)
        RCC_GPIOA_enable();
    if (Port == GPIOB)
        RCC_GPIOB_enable();
    if (Port == GPIOC)
        RCC_GPIOC_enable();

    //[TO-DO] YOUR CODE GOES HERE
    // Make it for GPIOB, GPIOD..GPIOH

    // You can also make a more general function of
    void RCC_GPIO_enable(GPIO_TypeDef *Port);

    GPIO_mode(pinName, mode);
}


// GPIO Mode          : Input(00), Output(01), AlterFunc(10), Analog(11)
void GPIO_mode(PinName_t pinName, uint32_t mode){
   GPIO_TypeDef * Port;
   unsigned int pin;
   ecPinmap(pinName,&Port,&pin);
   Port->MODER &= ~(3UL<<(2*pin));
   Port->MODER |= mode<<(2*pin);
```

```c
}


// GPIO Speed          : Low speed (00), Medium speed (01), Fast speed (10), High
speed (11)
void GPIO_ospeed(PinName_t pinName, int speed){
    //[TO-DO] YOUR CODE GOES HERE
    GPIO_TypeDef* Port;
    unsigned int pin;
    ecPinmap(pinName, &Port, &pin);
    //[TO-DO] YOUR CODE GOES HERE
    Port->OSPEEDR &= ~(3UL << (2 * pin));
    Port->OSPEEDR |= speed<<(2 * pin);
}

// GPIO Output Type: Output push-pull (0, reset), Output open drain (1)
void GPIO_otype(PinName_t pinName, int type){
    //[TO-DO] YOUR CODE GOES HERE
    GPIO_TypeDef* Port;
    unsigned int pin;
    ecPinmap(pinName, &Port, &pin);
    //[TO-DO] YOUR CODE GOES HERE
    Port->OTYPER &= ~(1 << (pin));
    Port->OTYPER |= type << (pin);
}

// GPIO Push-Pull    : No pull-up, pull-down (00), Pull-up (01), Pull-down (10),
Reserved (11)
void GPIO_pupd(PinName_t pinName, int pupd){
    //[TO-DO] YOUR CODE GOES HERE
    GPIO_TypeDef* Port;
    unsigned int pin;
    ecPinmap(pinName, &Port, &pin);
    //[TO-DO] YOUR CODE GOES HERE
    Port-> PUPDR &= ~(3UL << (2 * pin));
    Port-> PUPDR |= pupd << (2 * pin);
}

int GPIO_read(PinName_t pinName){
    //[TO-DO] YOUR CODE GOES HERE
    GPIO_TypeDef* Port;
    unsigned int pin;
    ecPinmap(pinName, &Port, &pin);
    //[TO-DO] YOUR CODE GOES HERE
    return (Port->IDR >> pin) & (1UL); //[TO-DO] YOUR CODE GOES HERE
}

void GPIO_write(PinName_t pinName, int Output) {
    GPIO_TypeDef* Port;
    unsigned int pin;
    ecPinmap(pinName, &Port, &pin);
    Port->ODR &= ~(1 << (pin));
    Port->ODR |= Output << (pin);
}

void GPIO_toggle(PinName_t pinName) {
    GPIO_TypeDef* Port;
    unsigned int pin;
```

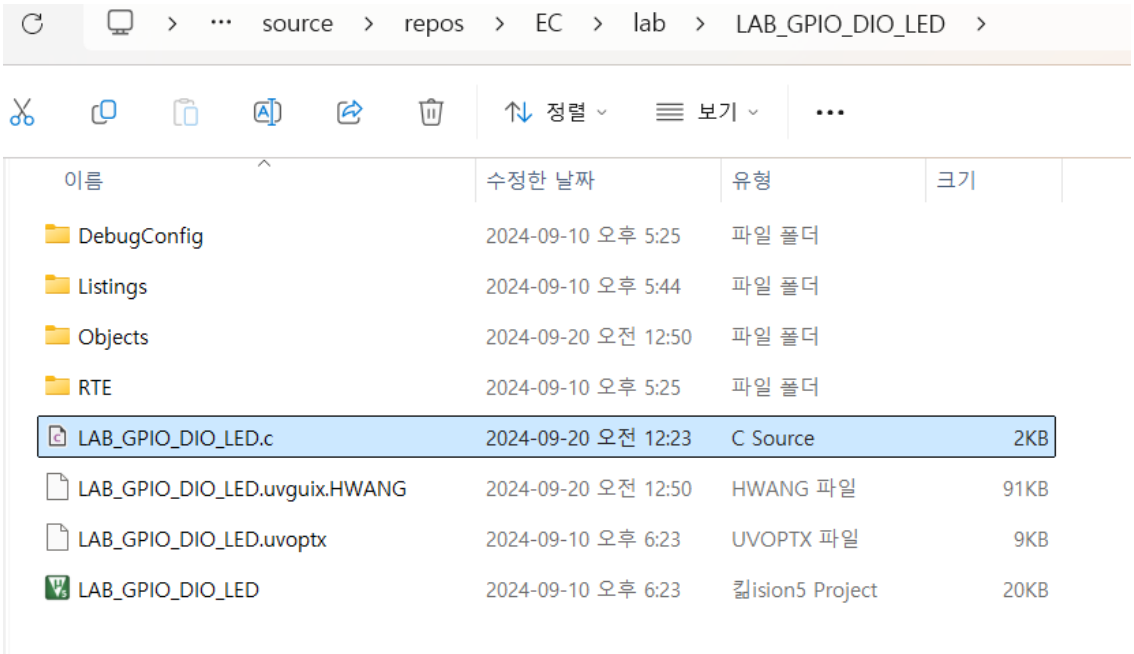
```
    ecPinmap(pinName, &Port, &pin);
    Port->ODR ^= 1 << (pin);
}

void delay_ms(int num) {
for(int i = 0; i < num*100 ;i++){}
}
```

Problem 2: Toggle LED with Button

Procedure

1. Create a new project under the directory `\repos\EC\LAB\`

- The project name is "**LAB_GPIO_DIO_LED".**
- Name the source file as "**LAB_GPIO_DIO_LED.c"**



2. Include your library **ecGPIO2.h, ecGPIO2.c** in `\repos\EC\lib\`.

3. Toggle the LED by pushing the button.

- Push button (LED ON), Push Button (LED OFF) and repeat

Configuration

Button (B1)	**LED**
Digital In	Digital Out
GPIOC, Pin 13	GPIOA, Pin 5
PULL-UP	Open-Drain, Pull-up, Medium Speed

Main code

```c
#include "stm32f4xx.h"
#include "ecRCC2.h"
#include "ecGPIO2_student.h"

PinName_t LED_pin = PA_5;
PinName_t button_pin = PC_13;

void setup(void);

int main(void) {
    // Initialiization -------------------------------------------------------
    setup();
    int lastButtonState = HIGH;
    GPIO_write(LED_pin, LOW);
    // Inifinite Loop ---------------------------------------------------------
    while(1){

        int buttonState = GPIO_read(BUTTON_PIN);

        if (buttonState == LOW && lastButtonState == HIGH) {
            GPIO_toggle(LED_pin);
        }
    lastButtonState = buttonState;
    }
}

// Initialiization
void setup(void)
{
    RCC_HSI_init();
    GPIO_init(button_pin, INPUT);
    GPIO_init(LED_pin, OUTPUT);
    GPIO_pupd(LED_pin, PULL_UP);
    GPIO_pupd(button_pin, PULL_UP);
    GPIO_otype(LED_pin, 0);
    GPIO_ospeed(LED_pin, 1);
}
```

Discussion

1. Find out a typical solution for software debouncing and hardware debouncing.

   For software debouncing, we can use ***'wait and see'*** technique and ***'counter debouncer'*** technique. For those technique implementation is important. There for at those techniques, using timer interrupts is better than Loop-based time delay.

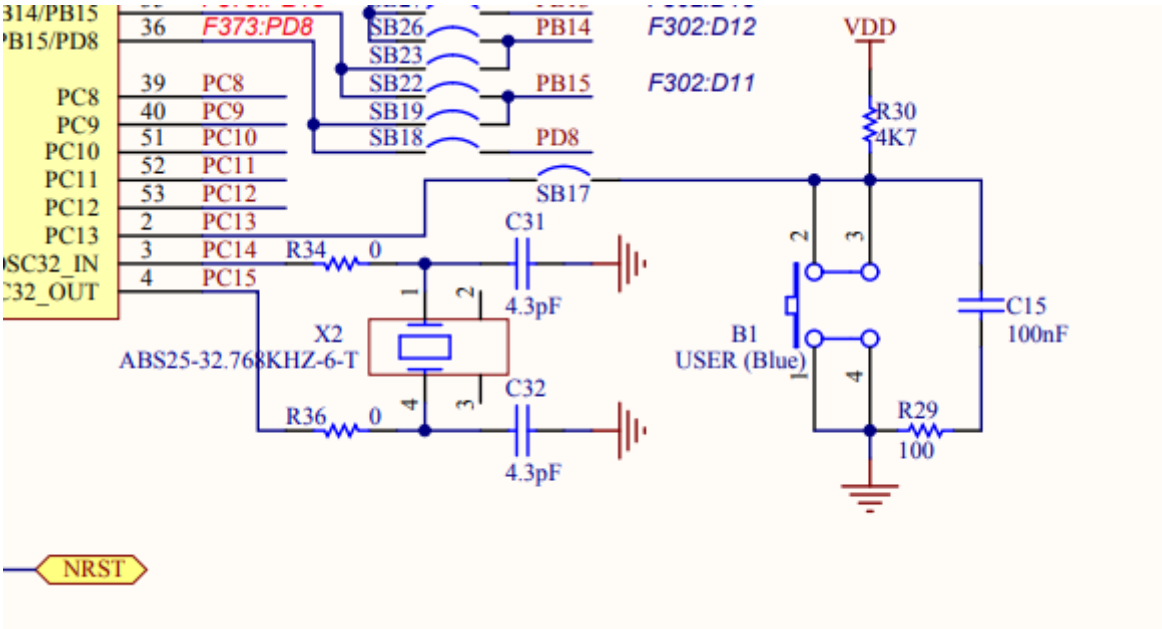
   For hardware debouncing, we use a simple ***'RC circuit that acts as a low-pass filter'***.

   No software debouncing is used when it has been debounced by hardware.

2. What method of debouncing did this NUCLEO board use for the push-button(B1)?

   Finding out by PCB Schematic, this NUCLEO-F411RE board use **hardware debouncing** by RC low-pass filter.

   There for we do not need software debouncing.



Problem 3: Toggle LED with Button

Procedure

1. Create a new project under the directory `\repos\EC\LAB\`

- The project name is "**LAB_GPIO_DIO_multiLED".**
- Name the source file as "**LAB_GPIO_DIO_multiLED.c"**

2. Include your library **ecGPIO2.h, ecGPIO2.c** in `\repos\lib\`.
3. Connect 4 LEDs externally with necessary load resistors.

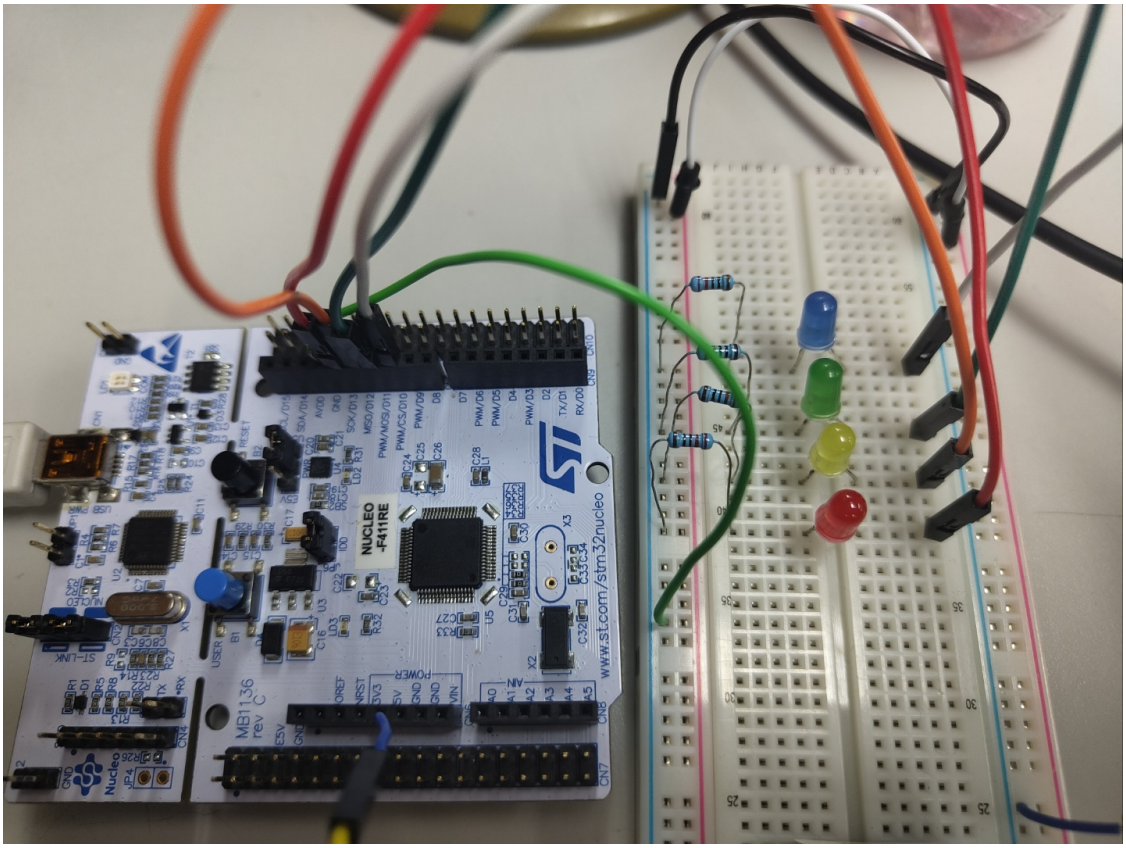
- As Button B1 is Pressed, light one LED at a time, in sequence.
- Example: LED0--> LED1--> ...LED3--> ...LED0....

Configuration

Button	LED
Digital In	Digital Out
GPIOC, Pin 13	PA5, PA6, PA7, PB6
PULL-UP	Push-Pull, Pull-up, Medium Speed

Circuit Diagram

Circuit diagram



Main Code

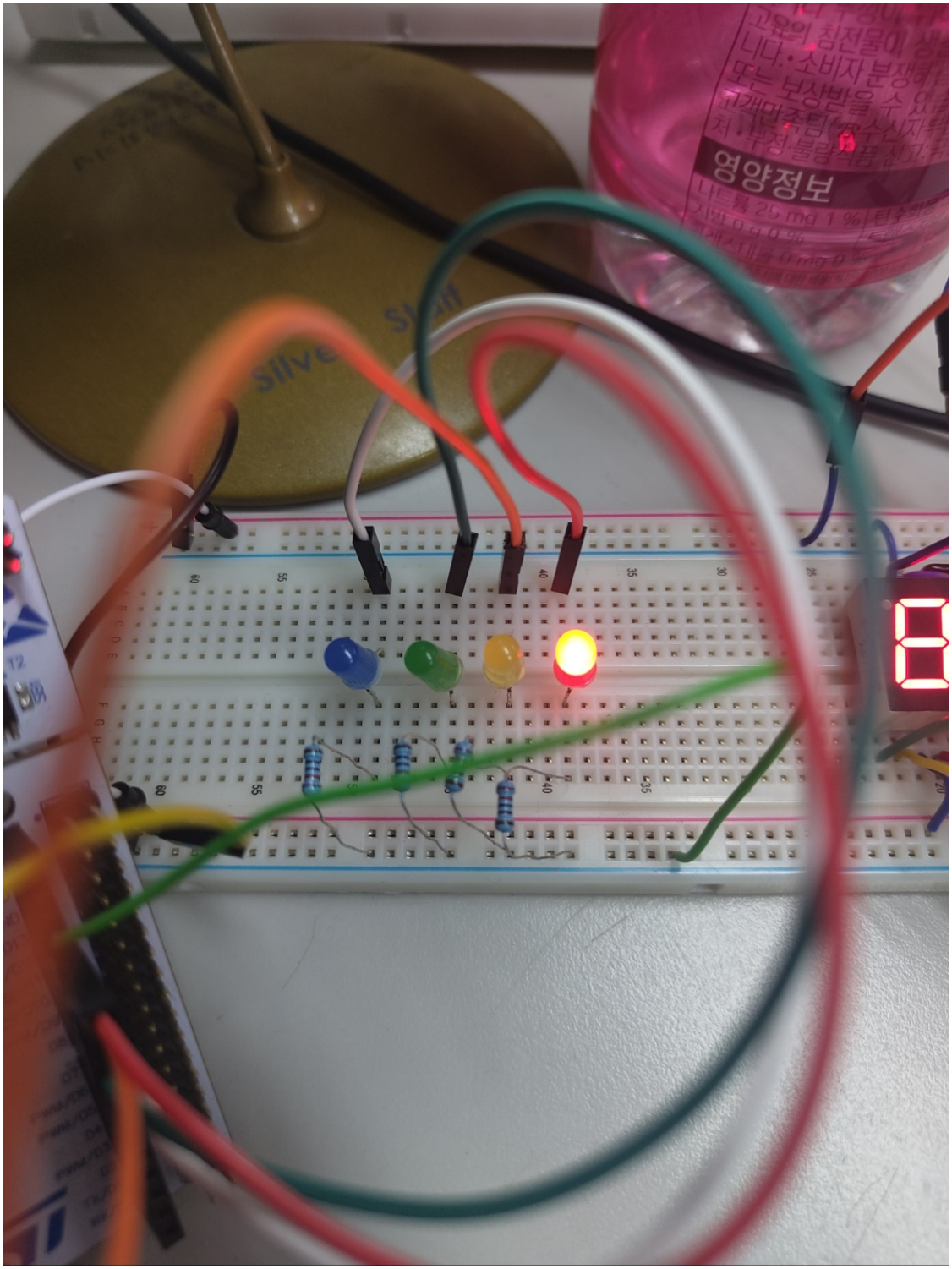
```
#include "stm32f4xx.h"
#include "ecRCC2.h"
#include "ecGPIO2_student.h"

PinName_t LED_pin1 = PA_5;
PinName_t LED_pin2 = PA_6;
PinName_t LED_pin3 = PA_7;
PinName_t LED_pin4 = PB_6;

PinName_t button_pin = PC_13;

void setup(void);
```

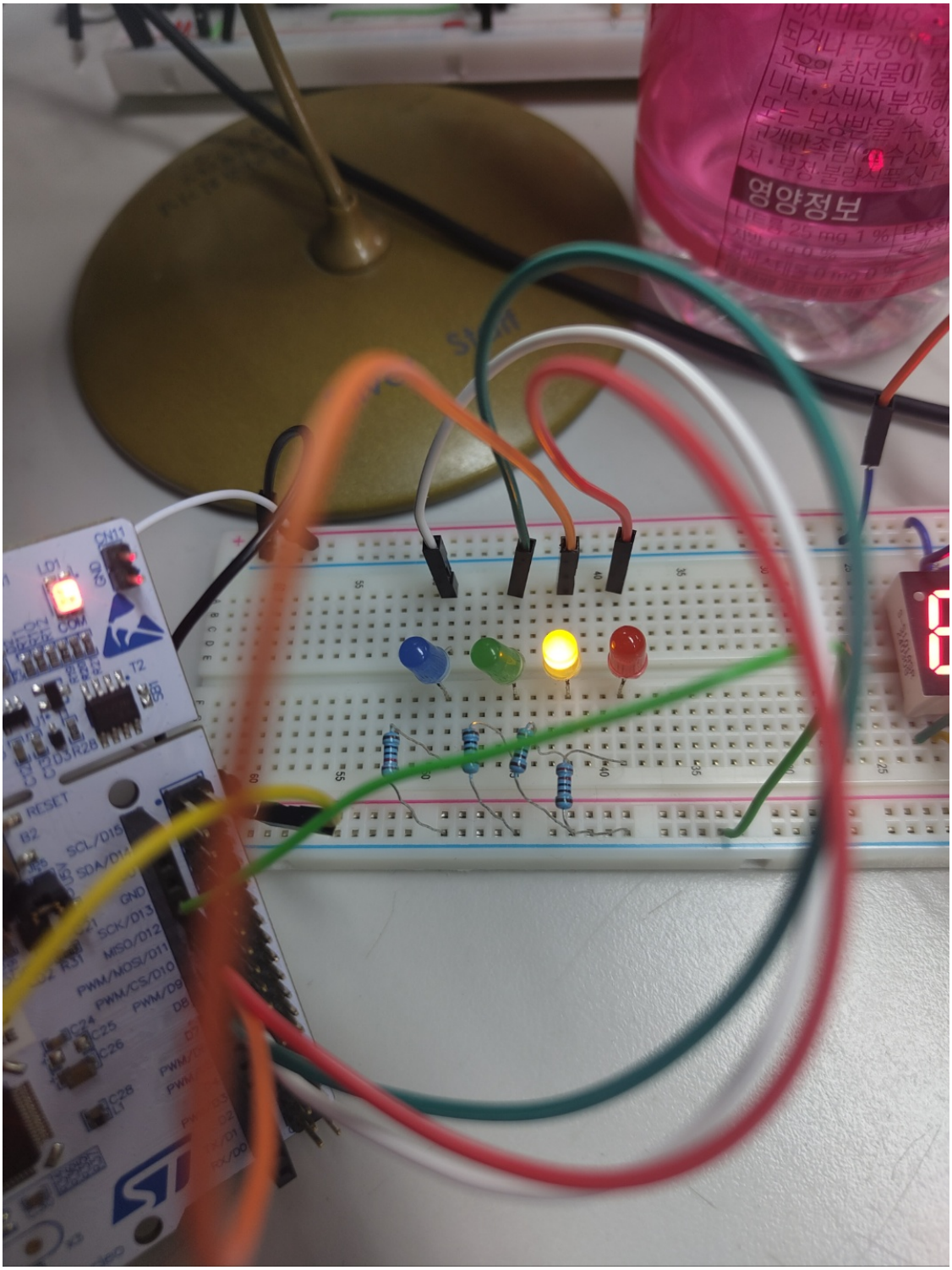
```
int main(void) {
    // Initialiization ------------------------------------------------------------
    setup();
    int lastButtonState = HIGH;
    GPIO_write(LED_pin1 , LOW);
    GPIO_write(LED_pin2 , LOW);
    GPIO_write(LED_pin3 , LOW);
    GPIO_write(LED_pin4 , LOW);
    // Inifinite Loop -------------------------------------------------------------
    while(1){

        int buttonState = GPIO_read(BUTTON_PIN);

        if(buttonState == LOW && lastButtonState == HIGH && GPIO_read(LED_pin1)
== LOW && GPIO_read(LED_pin2) == LOW && GPIO_read(LED_pin3) == LOW &&
GPIO_read(LED_pin4) == LOW)
            GPIO_toggle(LED_pin1);
        else if(buttonState == LOW && lastButtonState == HIGH &&
GPIO_read(LED_pin1) == HIGH){
            GPIO_toggle(LED_pin1);
            GPIO_toggle(LED_pin2);
        }
        else if (buttonState == LOW && lastButtonState == HIGH &&
GPIO_read(LED_pin2) == HIGH) {
            GPIO_toggle(LED_pin2);
            GPIO_toggle(LED_pin3);
        }
        else if (buttonState == LOW && lastButtonState == HIGH &&
GPIO_read(LED_pin3) == HIGH) {
            GPIO_toggle(LED_pin3);
            GPIO_toggle(LED_pin4);
        }
        else if (buttonState == LOW && lastButtonState == HIGH &&
GPIO_read(LED_pin4) == HIGH) {
            GPIO_toggle(LED_pin4);
            GPIO_toggle(LED_pin1);
        }
    lastButtonState = buttonState;
    }
}


// Initialiization
void setup(void)
{
    RCC_HSI_init();
    GPIO_init(button_pin, INPUT);  // calls RCC_GPIOC_enable()

    GPIO_init(LED_pin1, OUTPUT);    // calls RCC_GPIOA_enable()
    GPIO_pupd(LED_pin1, PULL_UP);
    GPIO_otype(LED_pin1, 0);

    GPIO_init(LED_pin2, OUTPUT);    // calls RCC_GPIOA_enable()
    GPIO_pupd(LED_pin2, PULL_UP);
    GPIO_otype(LED_pin2, 0);

    GPIO_init(LED_pin3, OUTPUT);    // calls RCC_GPIOA_enable()
```

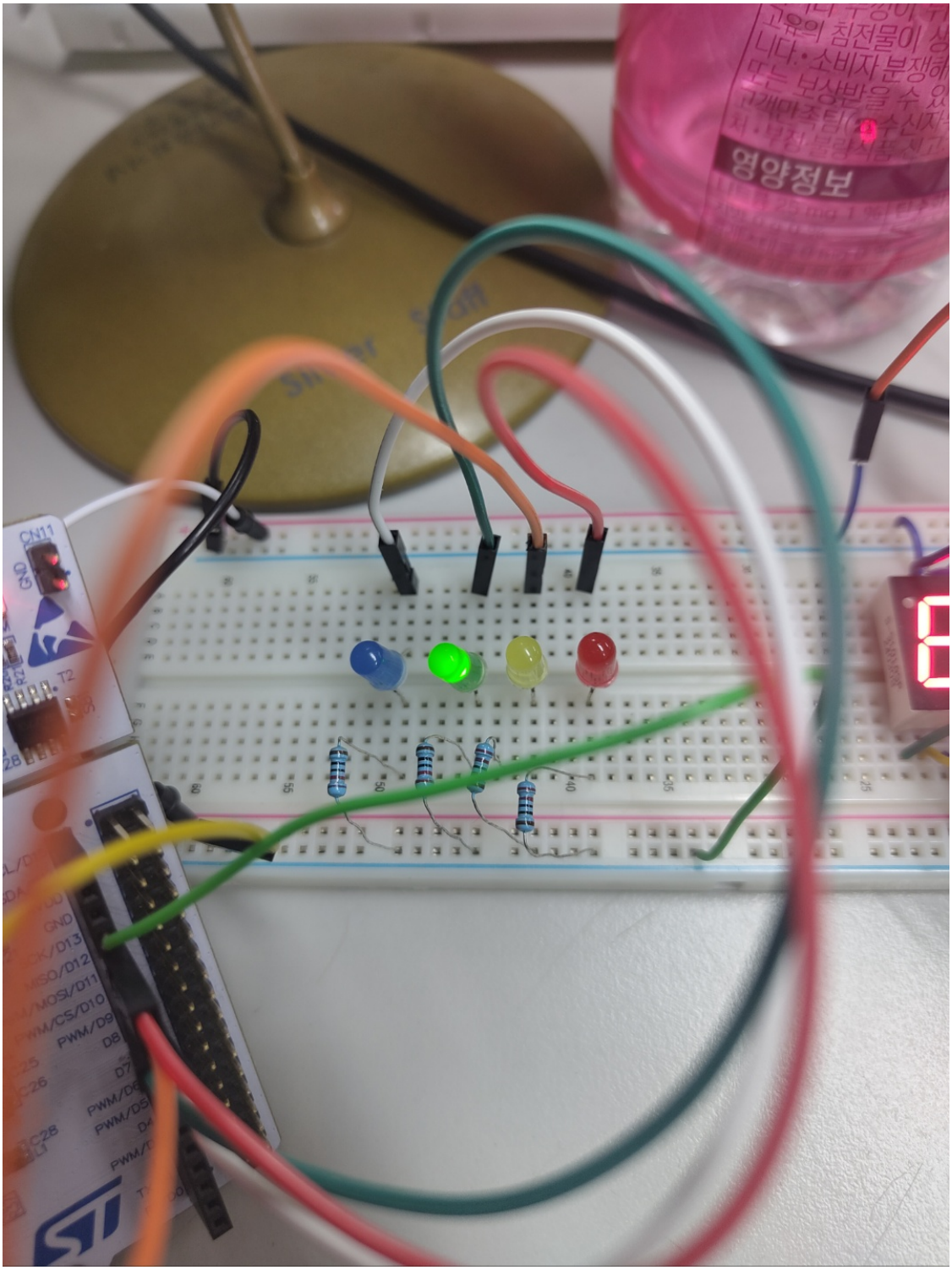
```
    GPIO_pupd(LED_pin3, PULL_UP);
    GPIO_otype(LED_pin3, 0);

    GPIO_init(LED_pin4, OUTPUT);     // calls RCC_GPIOA_enable()
    GPIO_pupd(LED_pin4, PULL_UP);
    GPIO_otype(LED_pin4, 0);
} CODE ONLY
```

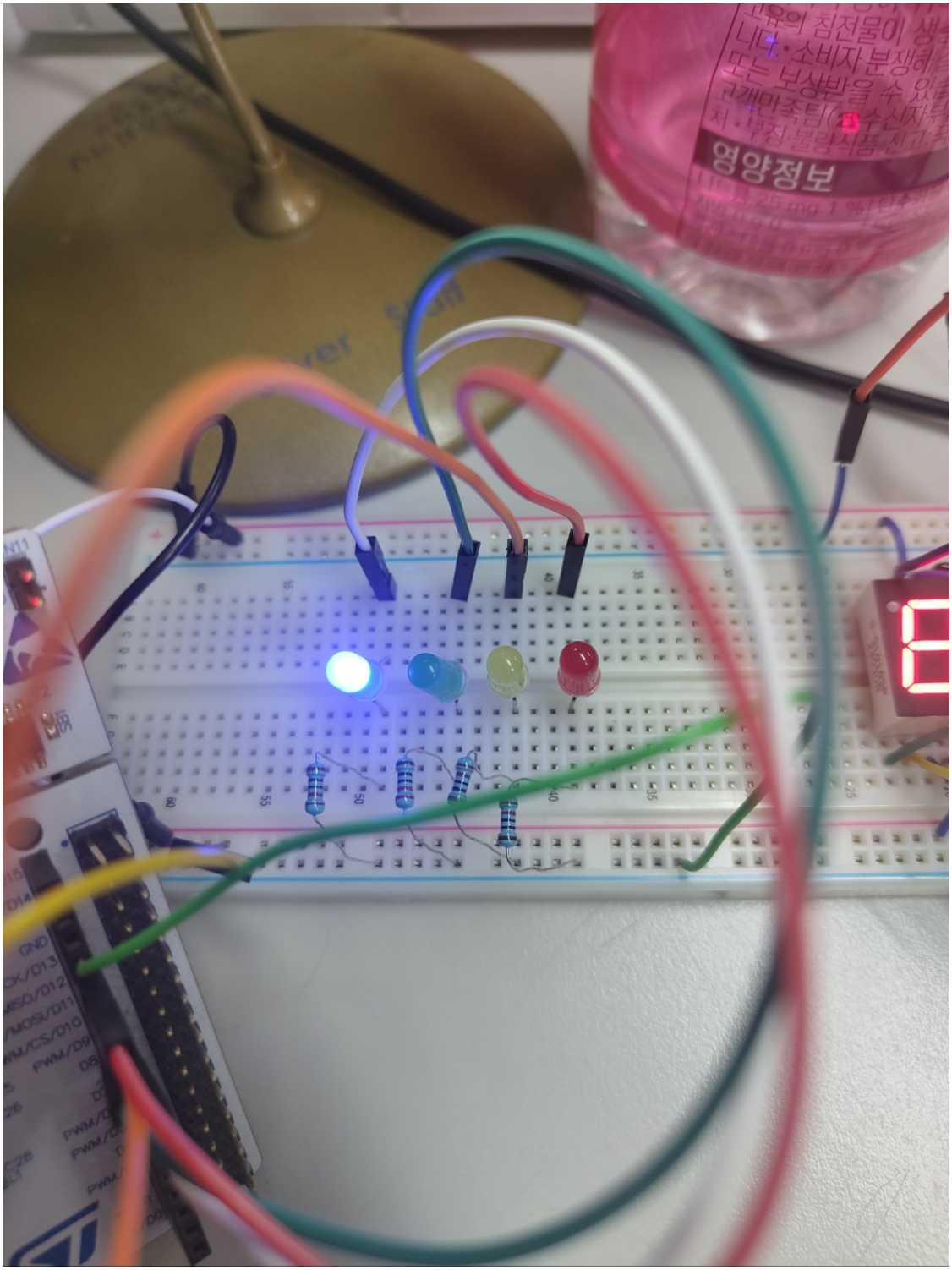
Results

Experiment images and results




영양정보
LD1
RESET
SCL/D15
SDA/D14
GND
SCK/D13
MISO/D12
PWM/MOSI/D11
PWM/CS/D10
PWM/D9


영양정보
CN11
GND
T2
GND
SCK/D13
MISO/D12
MOSI/D11
PWM/CS/D10
PWM/D9
D8
D7
PWM/D6
PWM/D5
D4



demo video link

Discussion

1. Find out a typical solution for software debouncing and hardware debouncing. What method of debouncing did this NUCLEO board use for the push-button(B1)?

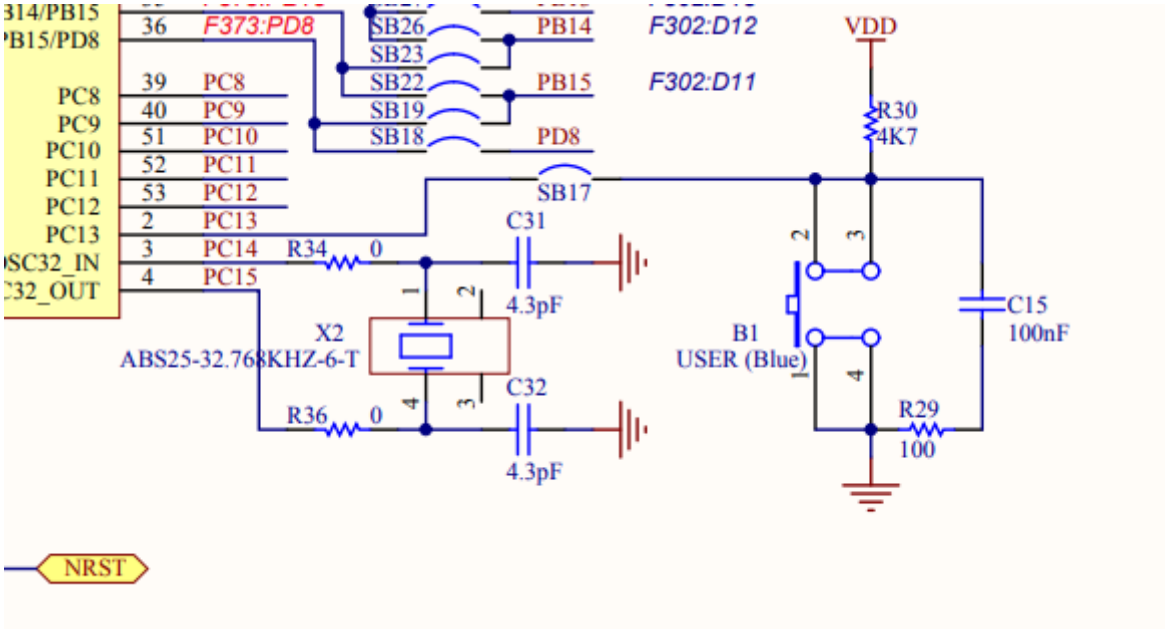
   For software debouncing, we can use ***'wait and see'*** technique and ***'counter debouncer'*** technique. For those technique implementation is important. There for at those techniques, using timer interrupts is better than Loop-based time delay.

   For hardware debouncing, we use a simple ***'RC circuit that acts as a low-pass filter'***.

   No software debouncing is used when it has been debounced by hardware.

Finding out by PCB Schematic, this NUCLEO-F411RE board use **hardware debouncing** by RC low-pass filter.

There for we do not need software debouncing.



Reference

Complete list of all references used (github, blog, paper, etc)

> 1. Yifeng Zhu - Embedded Systems with ARM Cortex-M Microcontrollers in Assembly Language and C (2018)
> 2. NUCLEO-F411RE PCB schematic File

Troubleshooting

When setting Output to Open-Drain mode the LED was darker then Push-Pull mode. The reason of this is Push-pull mode allows the pin to supply and absorb current. On the other hand, Open-Drain mode can only absorb current.

The benefit of Open-drain is to connect directly several outputs together. Compared to Open-drain, push-pull mode has a faster speed.